Procedural Island Circuit

CIS 5660 Final Project

Team Member: Peizhi Zhang

Introduction

Race circuits combine a simple organizing structure—a continuous loop with rich geometric and visual variation. Changing the arrangement of straights and corners produces different layouts, while curbs, barriers, vegetation, and terrain turn those layouts into complete environments.

This project explores how procedural rules can generate various racing environments. I plan to develop a generator that creates a closed circuit from a random seed, checks its geometric validity, constructs the road and trackside details, and places the result on a stylized island. The visual direction is a colorful miniature racing diorama with silhouettes and a restrained material palette.

Goal

The goal is to generate multiple visually distinct, geometrically closed circuits and present them as polished island scenes. The main technical contribution will be the pipeline connecting seeded layout generation, constraint validation, road mesh construction, and road-aware environment placement.

The core deliverable is an environment generator with an overview camera and an animated track-following camera. Geometric validation will check layout and mesh constraints; it will not certify handling for a particular vehicle or measure racing difficulty.

Criteria

- Reproduce the same track and decoration from the same seed and settings within the same implementation version.
- Produce a continuous, constant-width road with a clean closing seam, no detected nonlocal overlaps, and no visibly folded road or curb triangles.
- Keep props outside the road and its safety margin, accounting for prop footprints.
- Support parameter changes, an overview camera, and a camera that completes a smooth lap.

Inspiration / Reference

Visual reference: Kenney — Racing Kit



Reference images by Kenney, from the creator's Racing Kit post on OpenGameArt, listed as CC0. These are reference images, not project results. Image display requires an internet connection.

This project will use this reference for simplified road furniture, readable borders, and a cohesive low-poly visual vocabulary. The island setting is my proposed composition. The reference's modular track pieces will not replace the generated centerline and road mesh. Initial props will use simple primitives; any external assets adopted later will be credited separately.

Technical references

1. Three.js: CatmullRomCurve3. Reference for a closed centripetal Catmull–Rom spline.
2. Three.js: Curve. Reference for arc-length lookup, approximately equal-distance sampling, and tangent evaluation.
3. Three.js: InstancedMesh. Reference for rendering repeated props with shared geometry and materials.
4. Gustavo Maciel, 2013. Generating Procedural Racetracks.
5. Cem Yuksel, Scott Schaefer, and John Keyser, 2011. Parameterization and Applications of Catmull-Rom Curves.

6. Amit Patel, Red Blob Games. Making Maps with Noise Functions.

Specification

Core features

Feature	Planned behavior
Seeded layout	Generate a single closed, planar track with varied corner arrangements.
Shape controls	Expose seed, overall scale, curve complexity, and road width within supported ranges.
Validation	Reject intersections, insufficient clearance, overly tight turns, and invalid road geometry.
Road geometry	Generate a continuous road ribbon with distance-based UVs and alternating curb colors.
Trackside details	Generate barriers, posts, and a start/finish marking from shared track samples.
Island environment	Build a low-poly island with a flat road corridor, surrounding relief, trees, rocks, and a static water plane.
Presentation	Provide orbit/overview and track-following camera modes, with pause and restart controls.
Debug view	Show control points, sampled centerline, rejected regions, and validation statistics.

Curve complexity will control control-point count and radial variation. It is a shape parameter rather than a promise of driving difficulty. Overall scale controls the layout footprint rather than specifying an exact lap length.

Techniques

1. Seeded closed-curve generation

A seeded pseudorandom generator will place approximately 10 control points in angular order around a circle or ellipse. Bounded angular perturbations and radial variation will produce different layouts. A closed centripetal Catmull–Rom spline will interpolate these points.

Angular ordering is only an initialization strategy: it does not guarantee that the smoothed curve or finite-width road is valid. Separate deterministic random streams for layout and decoration will prevent prop changes from unexpectedly changing the track.

2. Arc-length sampling and road construction

This project will approximate cumulative arc length and resample the curve at roughly uniform distances. Each sample will store position, tangent, lateral direction, distance along the loop, and estimated curvature.

Adjacent cross-sections will form indexed triangles. The same frames will generate curbs and trackside elements. Longitudinal UV coordinates will use distance along the curve. Duplicated closing vertices with matching positions/normals and separate UVs will allow a clean geometric seam without a large UV jump inside a triangle.

3. Road-aware island and object placement

The island boundary will enclose the validated road's bounds with an additional margin. The core track will remain at a fixed elevation. Surrounding low-frequency terrain variation will blend toward that elevation near the road, leaving a level corridor and avoiding terrain clipping through the asphalt. A static water plane and simple shoreline palette will establish the island setting.

Trees and rocks will use Poisson disk sampling for minimum spacing. Candidates will then be rejected if their footprints intersect the road corridor, shoreline exclusion region, or reserved structures. Barriers and posts will follow regular distances along the track. The start/finish structure will use its selected local frame. Repeated props will use instanced rendering where appropriate.

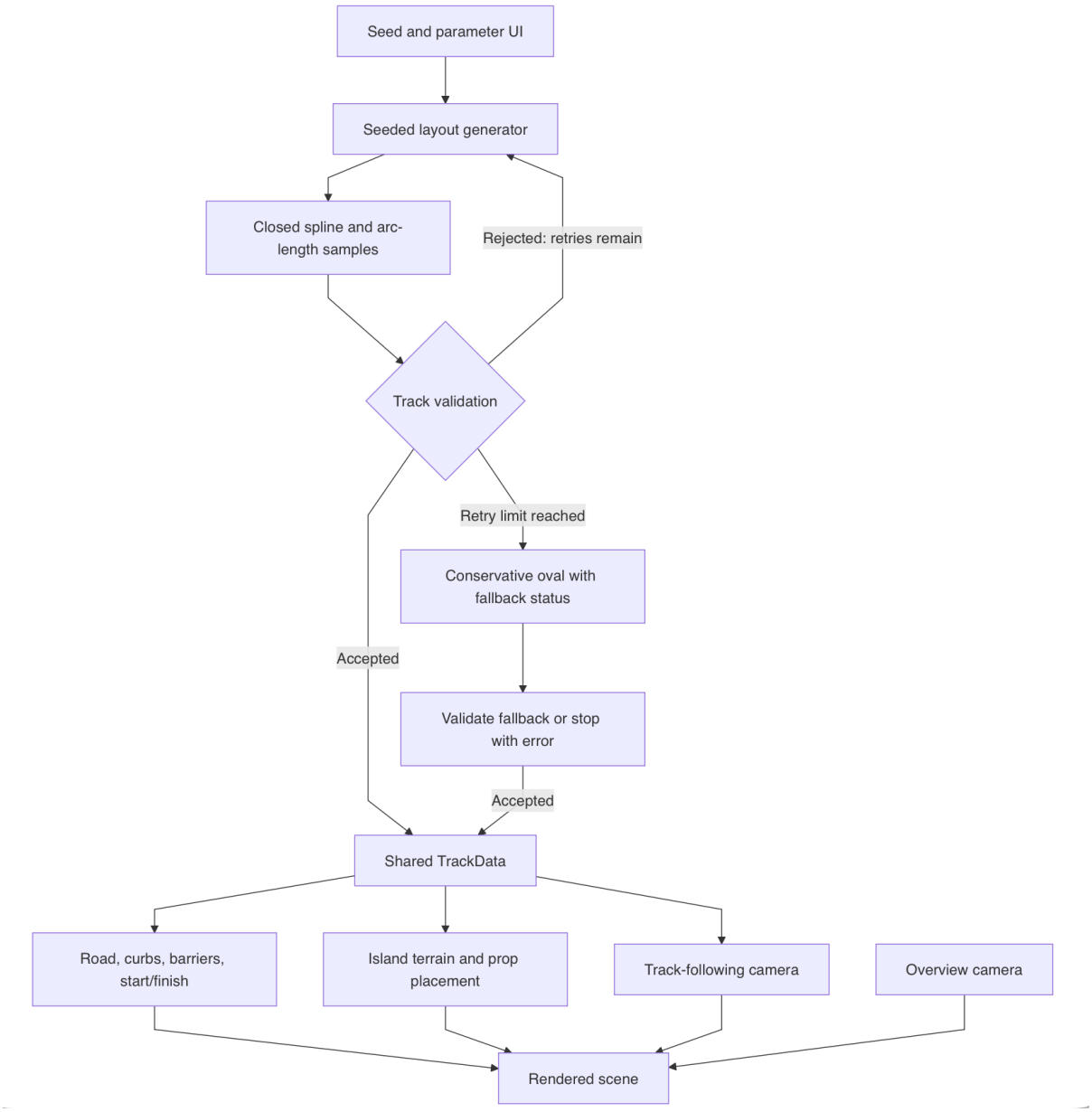
4. Track-following camera

The camera will advance by distance along the closed curve instead of by raw spline parameter to reduce unwanted speed variation. Its position will be offset above the track, and its target will look ahead along the curve. Smoothed orientation and cyclic distance wrapping will reduce abrupt turns and closure jumps.

Design

Proposed implementation

The proposed stack is TypeScript, Three.js, and a lightweight parameter UI. Three.js will handle rendering and provide spline utilities; project code will implement layout rules, validity checks, road construction, terrain integration, and placement logic. The platform can be revised before implementation if an existing course framework is more suitable.



Module	Function
LayoutGenerator	Seeded control points, candidate retries, and fallback layout.
TrackSampler	Spline, arc-length table, local frames, and curvature estimates.
TrackValidator	Intersection, clearance, curvature, and ribbon checks.
RoadBuilder	Road mesh, curbs, UVs, barriers, and start/finish details.
EnvironmentBuilder	Island mesh, road corridor, and prop distribution.
SceneController	Rendering, parameter updates, camera modes, and diagnostics.

The shared `TrackData` structure will contain the seed/settings, accepted control points, sampled frames, total length, road width, start/finish distance, and validation report. Geometry and camera modules will consume this representation to remain aligned. Regeneration will replace and dispose of previous geometry and instance buffers.

Timeline

Period / deadline	Responsibilities	Deliverable
Oct. 31–Nov. 4 — Design Doc	Confirm scope with instructor/TA; design doc.	Approved scope and design document submitted.
Nov. 5–11 — Milestone 1	Implement sampling, core checks, bounded retries, fallback, road/curb meshes, and parameter controls. Add crude island, barrier, prop, and camera versions so most generator functionality exists.	Working generator; several seed outputs; bug fix.
Nov. 12–18	Strengthen closure handling and geometry checks; integrate terrain and the road corridor; refine props, start/finish placement, and camera motion. Run fixed-seed and parameter-boundary checks.	Stable core feature set, diagnostic results, and an initial polished scene.
Nov. 19–23 — Milestone 2	Finish core features, resolve failures, tune supported parameter ranges, and prepare the classroom demo. Attempt an extension only if the core is stable.	Feature-complete working copy; detailed images/video and report.
Nov. 24–30 — Final	Incorporate critique; polish lighting, materials, colors, and camera motion; select varied seeds; record the demo and document results and limitations.	Final code, image gallery, video, live demo, presentation.